

A complete deep-dive into ES6 classes — class syntax, constructors, instance properties, private fields, getters/setters, inheritance, abstract classes, and the prototype system underneath.

Part 1 — What is OOP?

Object-Oriented Programming is a programming paradigm that organizes code around **objects** — bundles of related data (properties) and behavior (methods).

Core concepts:

- **Encapsulation** — hide internal details, expose a clean interface
 - **Abstraction** — hide complexity, show only what's necessary
 - **Inheritance** — child classes reuse and extend parent class behavior
 - **Polymorphism** — different classes respond to the same method differently
-

Part 2 — Defining a Class

Two Syntaxes

```
// Declaration (like function declaration)
class Person {
  // ...
}

// Expression (like function expression - can be anonymous or named)
const Person = class {
  // ...
};
```

Classes are NOT hoisted like `var`. You cannot use a class before it's defined.

Part 3 — Class Anatomy

```
class Person {

    // --- Public instance field (defined on each instance) ---
    email = '';

    // --- Private field (# prefix - truly private, inaccessible
    outside) ---
    #balance;

    // --- Constructor --- (runs when 'new Person(...)' is called)
    constructor(id, firstName, lastName, age, balance, balance2) {
        this.id          = id;
        this.age          = age;
        this.firstName    = firstName;
        this.lastName     = lastName;
        this.balance      = balance2; // public property on instance
        this.#balance     = balance;  // private field - only accessible
inside class
    }

    // --- Method (on the prototype, shared across all instances) ---
    greet() {
        console.log(`Hello ${this.name}`);
    }

    // --- Getter ---
    get name() {
        return `${this.firstName} ${this.lastName}`;
    }

    // --- Setter ---
    set name(personName) {
        this._name = personName;
    }
}

const p1 = new Person(1, 'mona', 'mohamed', 20, 1000, 2000);
```

Part 4 — The constructor Method

The `constructor` is a special method that runs **automatically** when you create an instance with `new`. It sets up the initial state of the object.

```
const p1 = new Person(1, 'mona', 'mohamed', 20, 1000, 2000);
// Internally:
// 1. A new empty object is created: {}
// 2. 'this' inside constructor refers to that new object
// 3. Properties are assigned: this.id = 1, this.age = 20, etc.
// 4. The new object is returned automatically
```

Instance Properties vs Prototype Methods

```
class Person {
  constructor(name) {
    this.name = name; // INSTANCE property - unique per object,
    // stored ON the object

    // BAD - putting methods in constructor creates a new function for
    // EVERY instance
    // this.greet = function() { ... } ← wastes memory
  }

  greet() { // PROTOTYPE method - defined ONCE, shared by ALL
    instances
    console.log(`Hello ${this.name}`);
  }
}

const p1 = new Person("mona");
const p2 = new Person("omar");

// Both p1 and p2 share the SAME greet function (from prototype)
console.log(p1.greet === p2.greet); // true - same function reference
```

Memory efficiency: Methods defined in the class body go on

`Person.prototype` — one copy shared by all instances. Methods defined inside `constructor` with `this.method = ...` create a new copy for every instance.

Part 5 — Public Instance Fields

Declared at the top of the class body — they're set on each instance before the constructor runs:

```
class Person {
  email = ''; // every instance starts with email = ''
  role = 'user';

  constructor(name, email) {
    this.name = name;
    this.email = email; // overrides the default ''
  }
}

const p = new Person("mona", "mona@gmail.com");
console.log(p.email); // "mona@gmail.com"
```

Part 6 — Private Fields

Private fields are truly **inaccessible outside the class**. This is real encapsulation — not just a convention.

```
class Person {
  #balance; // declare private field

  constructor(balance) {
    this.#balance = balance;
  }

  getBalance() {
    return this.#balance; // accessible inside the class
  }

  deposit(amount) {
    if (amount > 0) this.#balance += amount;
  }
}
```

```
}

const p1 = new Person(1000);
console.log(p1.getBalance()); // 1000 - accessed via public method
console.log(p1.#balance);      // SyntaxError - truly private!
```

#balance vs balance in the Code

```
constructor(id, firstName, lastName, age, balance, balance2) {
  this.balance = balance2; // public - anyone can read/write
  this.#balance = balance; // private - only readable via methods
}
```

Both exist on the same object — `this.balance` is public, `this.#balance` is private with the actual sensitive value.

Part 7 — Getters and Setters

Getters and setters let you define **computed properties** — they look like properties from the outside but run code internally.

Getter

```
class Person {
  constructor(firstName, lastName) {
    this.firstName = firstName;
    this.lastName = lastName;
  }

  get name() {
    return `${this.firstName} ${this.lastName}`;
  }
}

const p = new Person("mona", "mohamed");
console.log(p.name); // "mona mohamed" - accessed like a property,
```

```
not a method call
// NOT: p.name() - no parentheses needed
```

Setter

```
class Person {
  constructor(firstName, lastName) {
    this.firstName = firstName;
    this.lastName = lastName;
  }

  set name(personName) {
    // Can validate, transform, or split before storing
    const parts = personName.split(' ');
    this.firstName = parts[0];
    this.lastName = parts[1] || '';
  }
}

const p = new Person("mona", "mohamed");
p.name = "omar ahmed"; // triggers setter - looks like assignment
console.log(p.firstName); // "omar"
console.log(p.lastName); // "ahmed"
```

Getter + Setter Together

```
class Temperature {
  #celsius;

  constructor(celsius) {
    this.#celsius = celsius;
  }

  get fahrenheit() {
    return this.#celsius * 9/5 + 32; // computed on the fly
  }

  set fahrenheit(f) {
    this.#celsius = (f - 32) * 5/9; // convert back when set
  }
}
```

```
}

const temp = new Temperature(0);
console.log(temp.fahrenheit); // 32
temp.fahrenheit = 212;
console.log(temp.fahrenheit); // 212
```

The `_name` convention in setters (like `this._name = personName`) is an old pattern to avoid infinite recursion when a getter and setter share a name with a stored property. Private fields `#` are the modern, correct solution.

Part 8 — Inheritance with `extends` and `super`

The Problem Without Inheritance

Without inheritance, every subtype duplicates shared code:

```
// Without inheritance - repetitive
class Car { constructor(brand) { this.brand = brand; } }
class Truck { constructor(brand) { this.brand = brand; } }
class Bike { constructor(brand) { this.brand = brand; } }
```

Inheritance — Share Common Code

```
class Vehicle {
  constructor(brand) {
    this.brand = brand;
  }

  start() {
    console.log(`${this.brand} engine starting...`);
  }
}

class Car extends Vehicle {
  constructor(brand, model) {
    super(brand); // MUST call super() first - calls Vehicle's
    constructor
  }
}
```

```

    this.model = model;
  }

  start() {
    console.log("Car engine start"); // overrides Vehicle's start
  }
}

const car1 = new Car('Toyota', '2026');
car1.start(); // "Car engine start"
console.log(car1.brand); // "Toyota" – set by Vehicle's constructor
via super
console.log(car1.model); // "2026"

```

super — Two Uses

```

// 1. super() in constructor – calls parent's constructor
constructor(brand, model) {
  super(brand); // must be called BEFORE using 'this'
  this.model = model;
}

// 2. super.method() in methods – calls parent's version
start() {
  super.start(); // calls Vehicle's start()
  console.log("Car-specific stuff");
}

```

Rule: If a child class has a `constructor`, it **must** call `super()` before accessing `this`. JavaScript will throw a `ReferenceError` otherwise.

Part 9 — Abstract Classes (Pattern)

JavaScript has no built-in `abstract` keyword, but you can simulate it:

```

class Vehicle {
  constructor(brand) {
    // Prevent direct instantiation
  }
}

```



```

    if (this.constructor === Vehicle) {
        throw new Error("Cannot instantiate abstract class Vehicle");
    }
    this.brand = brand;
}

start() {
    // Force subclasses to implement their own start()
    throw new Error("start() must be implemented by subclass");
}
}

class Car extends Vehicle {
    constructor(brand, model) {
        super(brand);
        this.model = model;
    }

    start() {
        // Overrides the forced error – this is the required
        implementation
        console.log("Car engine start");
    }
}

const car1 = new Car('Toyota', '2026');
car1.start(); // "Car engine start" – works fine

const v1 = new Vehicle("Gelly"); // Error: Cannot instantiate
abstract class Vehicle

```

How `this.constructor === Vehicle` Works

`this.constructor` refers to the class that was actually used with `new`:

- `new Vehicle()` → `this.constructor === Vehicle` → throws
- `new Car()` → `this.constructor === Car` → passes, continues

Part 10 — Static Methods and Properties

Static members belong to the **class itself**, not to instances:

```
class MathHelper {
  static PI = 3.14159;

  static add(a, b) {
    return a + b;
  }
}

console.log(MathHelper.PI);          // 3.14159
console.log(MathHelper.add(2,3));    // 5
// NOT: new MathHelper().add(2,3) - statics don't belong to instances
```

Part 11 — The Prototype System (Under the Hood)

JavaScript classes are **syntactic sugar** over the prototype-based system. Understanding prototypes helps you understand how classes work internally.

```
class Person {
  constructor(name) { this.name = name; }
  greet() { console.log(`Hello ${this.name}`); }
}

const p = new Person("mona");

// What actually happens:
// p.__proto__ === Person.prototype → true
// p.greet === Person.prototype.greet → true

// When you call p.greet():
// JS looks for greet on p → not found
// Looks on p.__proto__ (Person.prototype) → found!
```

The Prototype Chain

```
p (instance)
  ↓ __proto__
Person.prototype ← methods live here
  ↓ __proto__
Object.prototype ← toString, hasOwnProperty, etc.
  ↓ __proto__
null              ← end of chain
```

Class vs `var` — Block Scoping Reminder

```
// var - function scoped, hoisted
var x = 10;
console.log(x); // 10

// let - block scoped, not accessible before declaration
let x = 10;
// console.log before this line → ReferenceError (TDZ)

// Classes behave like let - block scoped, not hoisted
```

Complete Summary

Class Syntax:

```
class Name { ... }      - declaration
const Name = class { ... } - expression
```

Constructor:

```
constructor() runs on 'new'
Sets up instance state
Methods belong on prototype - NOT in constructor
```

Fields:

```
email = ''      - public instance field (default value)
#balance        - private field (truly inaccessible outside class)
```

Getter/Setter:

`get prop()` - access like property, runs code
`set prop(val)` - assign like property, runs code

Inheritance:

`extends` - create child class
`super()` - call parent constructor (required first)
`super.method()` - call parent method

Abstract Pattern:

`if (this.constructor === Base) throw Error`
Forces subclasses to implement methods

Static:

`static method()` - belongs to class, not instances

Abdullah Ali

Contact : +201012613453